

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

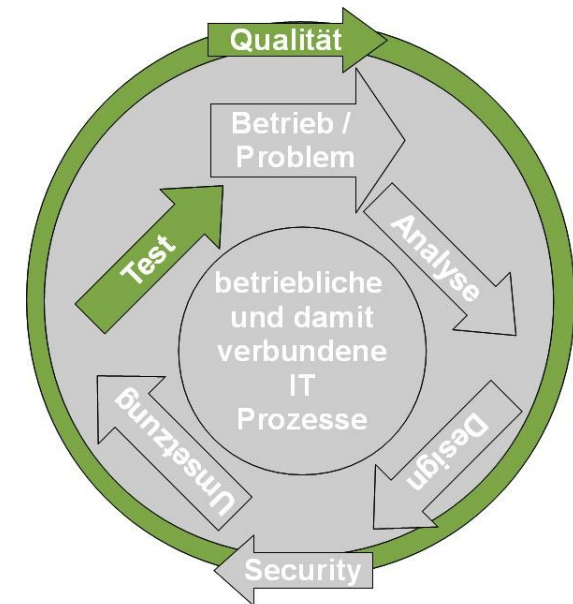
2.0: Qualitätsmanagement

v. 21.11.2016

Einordnung

2.0 Qualitätsmanagement

- Modellgetrieben SW-Entwicklung
- **Qualitätsmanagement**
- Testen



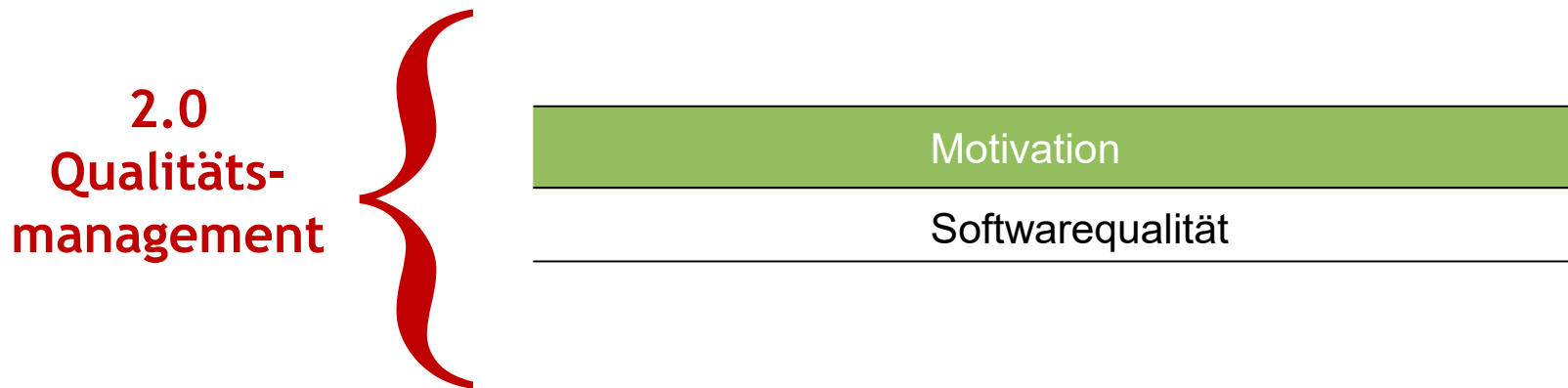
Basierend auf Beiträge von Prof. Sommerville (St. Andrews University), Prof. Martin Glinz (Universität Zürich) und dem Foliensatz „Basiswissen Softwaretest - Certified Tester“ des „German Testing Board“.

Literatur (s. Webseite):

- J. Ludewig, H. Lichter: Software Engineering - Grundlagen, Menschen, Prozesse, Techniken. Kapitel 5.

- Entwickeln des Verständnis was Qualität für Software bedeutet und wie sich die Qualität in Softwareprojekten gezielt messen und verbessern lässt.
- Unterscheiden von Qualitätsmaßnahmen für SE Prozesse und diese problemspezifisch auswählen, planen und anwenden können.

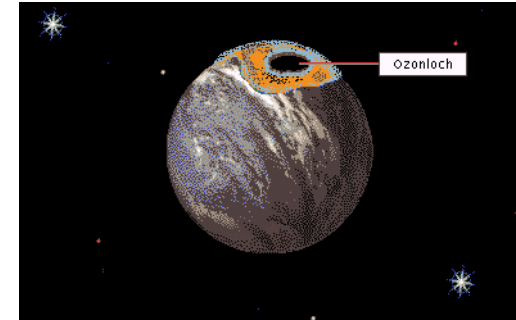
- **Letzter Abschnitt:** Technische Grundlagen für UML-Werkzeuge und MDA.
- **Dieses Kapitel 2:** Von Softwareentwicklung zu Softwareverifikation.
 - Dieser Abschnitt 2.0:
Kurze **Einführung** und **Motivation** für die Inhalte von Kapitel 2:
Qualitätsmanagement



Auswirkungen von Software-Fehlern

NASA - Erdbeobachtungssatelliten 1979-1985

- **Ozonloch** 7 Jahre (!) lang **nicht erkannt**.
- Ursache: Softwarefehler – Veränderung der Ozonschicht als Sensordrift durch automatische Nullpunktkorrektur »herausgemittelt«.



ESA, Kourou, Franz. Guyana, 4. Juni 1996

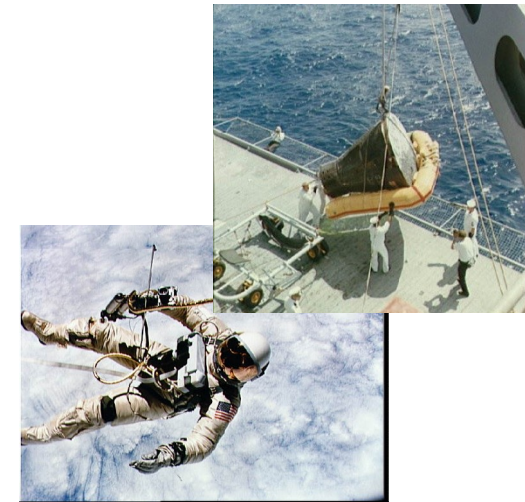
- **Selbstzerstörung der Ariane 5** beim Jungfernflug 39 Sekunden nach Start.
- Ursache: Softwarefehler – Lageregelungssoftware aus Ariane 4 ohne Test gegen Start-Trajektorie der Ariane 5 wiederverwendet, dadurch Konvertierungsfehler.

Ariane 5 Flight 501



Bemannte **NASA-Raumkapsel** Gemini V

- **Verfehlte ihren Landeplatz** um ca. 160 Kilometer.
- Ursache: Softwarefehler – Rotation der Erde um die Sonne nicht berücksichtigt !



21. April 2009: Software-Fehler legt T-Mobile Netz lahm:

- Millionen **T-Mobile-Kunden** waren **nicht erreichbar**.
- Nach wählen der Nummer kam von Frauenstimme:
„Dieser Anschluss ist aus technischen Gründen vorübergehend nicht erreichbar. Bitte rufen Sie später wieder an!“
- System wurde zurückgesetzt und neu gestartet.
- Ursache: **Softwarefehler** im Home Location Register (HLR).
→ Zuordnung der Telefonnummern zu einzelnen SIM-Karten.
- 3 Datenbanken mussten verfügbar gemacht werden.



NASA verliert auf Weg zur Venus befindliche **Raumsonde Mariner 1** am 22.6.1962:

»Because of a launch-vehicle deviation from the planned flight path, Mariner R-1 was destroyed by the range safety officer after approximately 290 seconds of flight.«

- **Korrekt** codiert wäre gewesen:
`DO 10 I=1,3` (Definition einer Schleife in FORTRAN IV)

...

`10 CONTINUE`

- **Fehlerhaft** codiert wurde

`DO10I=1.3`

...

`10 CONTINUE`

Welchen Effekt hat
dieser Fehler wohl?



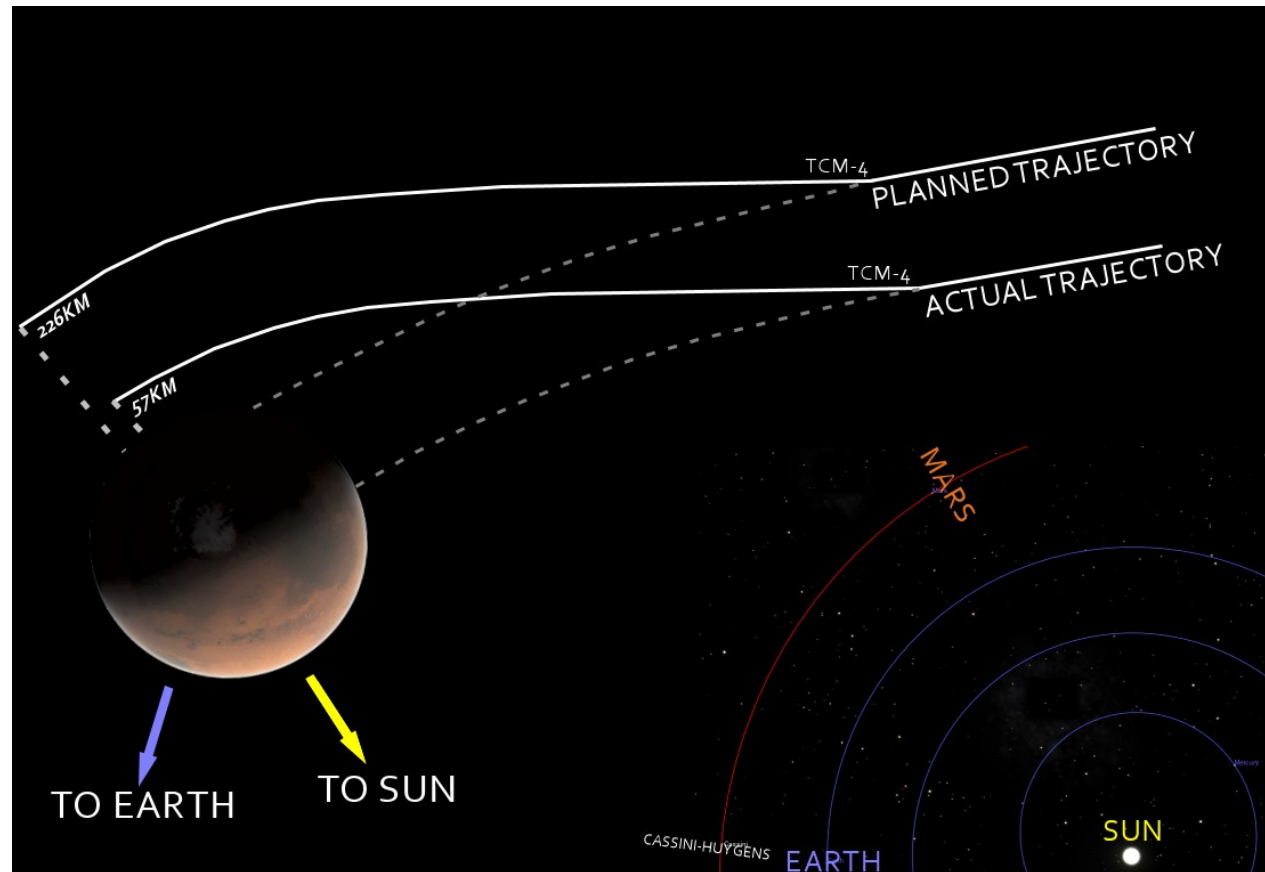
NASA verliert auf Weg zur Venus befindliche **Raumsonde Mariner 1** am 22.6.1962:

»Because of a launch-vehicle deviation from the planned flight path, Mariner R-1 was destroyed by the range safety officer after approximately 290 seconds of flight.«

- **Korrekt** codiert wäre gewesen:
DO 10 I=1,3 (Definition einer Schleife in FORTRAN IV)
...
10 CONTINUE
- **Fehlerhaft** codiert wurde
DO10I=1.3 (Zuweisung des Werts 1.3 an die Variable DO10I)
...
10 CONTINUE



Navigationsfehler zerstörte am 23. September 1999 die Mars Climate Orbiter Sonde, da Kontrollzentrum auf Erde Maßangaben in Feet verwendeten, die Software jedoch mit Metern rechnete.



Quelle: Wikimedia

- Speicherung eines Datums xx.yy.19zz als xx.yy.zz
- Nach 31. Dezember 1999 (Katastrophenszenarien!!!)
- xx.yy.zz interpretierbar als xx.yy.20zz
- Dienstleister und Berater haben **viel Geld** verdient an Y2K
- Einige bekannte Probleme:
 - United States Naval Observatory (offizieller Zeitmesser der USA) gab auf Website das Jahr 19100 an.
 - In Australien fielen Autobusfahrtscheinbestätigungsmaschinen aus.
 - Autofahrer erhielt Kfz-Steuerforderung für die letzten 100 Jahre über 760.000 US\$.

Gesamtaufwand für die Y2K-Projekte wurde
von Gartner auf weltweit „bis zu
600 Milliarden US-Dollar“
geschätzt.

2038 Problem / Unix Millennium Bug

Unix speichert Zeit als vorzeichenbehaftete 32 Bit-Zahl

- Interpretiert als Anzahl der Sekunden nach 00:00:00 UTC am 1. Januar 1970
- Letzte darstellbare Zeit 03:14:07 UTC am Dienstag, 19. Januar 2038 (2,147,483,647 Sekunden nach 1. Januar 1970) - danach Überlauf
- Verwundbare Systeme (viele!): MySQL, Unix, Linux, viele eingebettete Systeme

Programm mit 3 Integer-Eingabewerten testen.

(Annahme: Randbedingungen haben keinen Einfluss auf Testobjekt).

Wie viele Tests für alle möglichen Kombinationen von Eingaben ?

Wie lange dauert das Testen bei **100.000 Tests pro Sekunde** ?

- Mehr als 1 millisec ?
- Mehr als 1 sec ?
- Mehr als 1 min ?
- Mehr als 1 h ?
- ...

Programm mit 3 Integer-Eingabewerten testen.

(Annahme: Randbedingungen haben keinen Einfluss auf Testobjekt).

Wie viele Tests für alle möglichen Kombinationen von Eingaben ?

Wie lange dauert das Testen bei **100.000 Tests pro Sekunde** ?

- Jeder **Eingabewert** kann bei 16 Bit Integerzahlen 2^{16} unterschiedliche **Werte** annehmen.
- Bei drei unabhängigen Eingabewerten ergeben sich $2^{16} * 2^{16} * 2^{16} = 2^{48}$ Kombinationen.

=> **281.474.976.710.656 Testfälle.**

Dauer: ca. 90 Jahre.

Diskussion: Austesten auf Kontrollflussgraph ?

Idee: Maximale denkbare **Testmenge einschränken**.

- Durch Verwendung interner Informationen über Programm, z.B. Kontrollflussgraph.

Beispiel (s. Abb.): Testen eines Programms, das aus

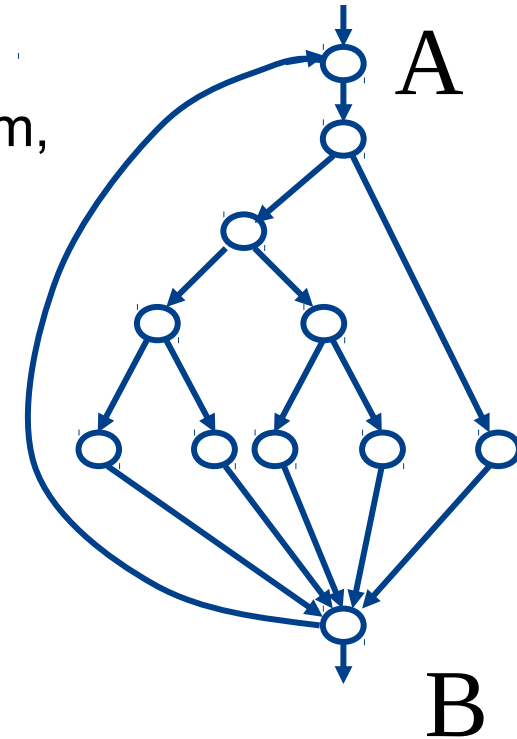
- **vier Verzweigungen** (IF-Anweisungen) und
- einer umfassenden **Schleife** besteht und
- **fünf mögliche Wege** im Schleifenrumpf enthält.

Annahmen:

- Verzweigungen voneinander **unabhängig**.
- **Max. 20** Schleifendurchläufe.

Wie viele Tests für alle möglichen Pfade im Kontrollflussgraph ?

Wie lange dauert das Testen bei **100.000 Tests pro Sekunde** ?



Mehr als 1 ms ?
Mehr als 1 sec ?
Mehr als 1 min ?
Mehr als 1 h ?
...

Diskussion: Austesten auf Kontrollflussgraph ?

Idee: Maximale denkbare **Testmenge einschränken**.

- Durch Verwendung interner Informationen über Programm, z.B. Kontrollflussgraph.

Beispiel (s. Abb.): Testen eines Programms, das aus

- **vier Verzweigungen** (IF-Anweisungen) und
- einer umfassenden **Schleife** besteht und
- **fünf mögliche Wege** im Schleifenrumpf enthält.

Annahmen:

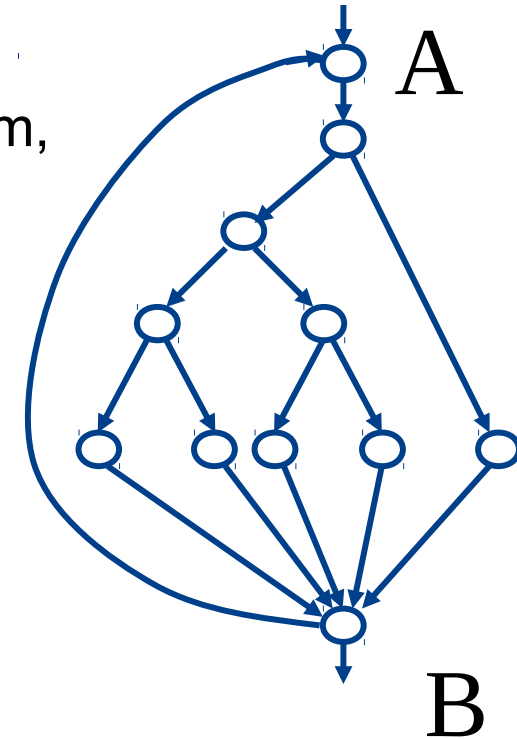
- Verzweigungen voneinander **unabhängig**.
- **Max. 20** Schleifendurchläufe

Wie viele Tests für alle möglichen Pfade im Kontrollflussgraph ?

Wie lange dauert das Testen bei **100.000 Tests pro Sekunde** ?

→ **Rechnung:** $5^1 + 5^2 + \dots + 5^{18} + 5^{19} + 5^{20}$

- **Es sind 119.209.289.550.780 Testfälle. Dauer: ca. 38 Jahre.**



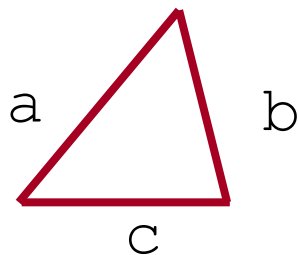
Diskussion: Testen auf Fallunterscheidungen ?

Weitere Idee, um Testaufwand zu begrenzen: **Fallunterscheidungen.**

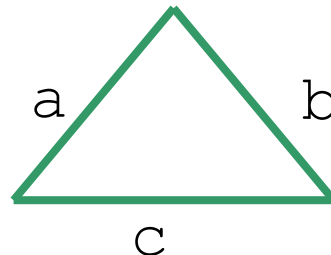
- Dann z.B. je ein Test pro Fallunterscheidung.

Beispiel: Programm testen, das

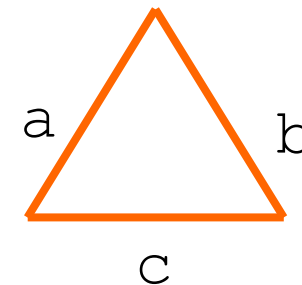
- 3 ganzzahlige positive **Werte** einliest,
- sie als **Längen** eines Dreiecks interpretiert und
- ausgibt, ob Dreieck **ungleichseitig**, **gleichschenkelig** oder **gleichseitig** ist.



$a \neq b$ und
 $b \neq c$ und
 $a \neq c$



$a = b \neq c$ oder
 $a \neq b = c$ oder
 $a = c \neq b$

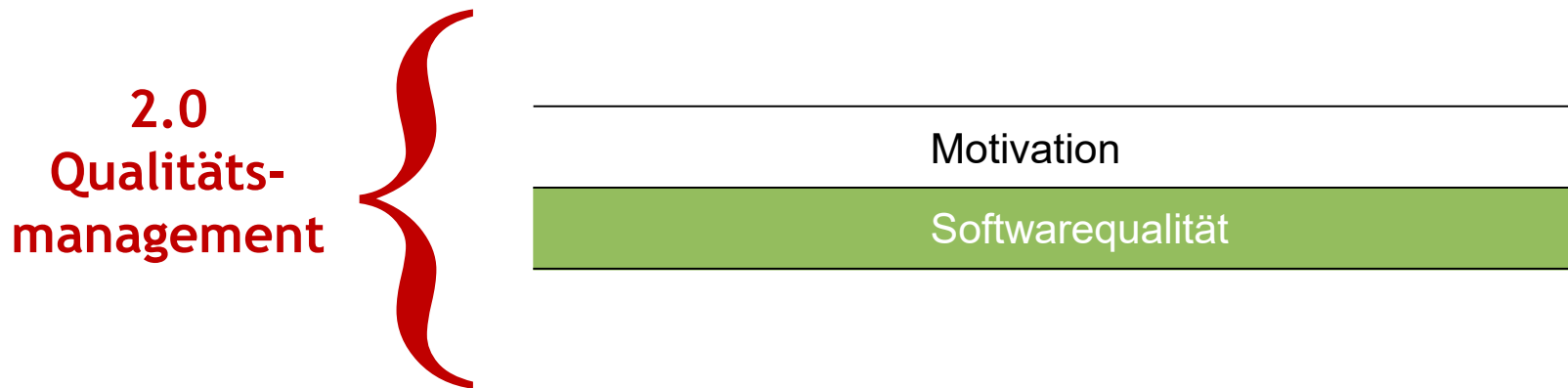


$a = b = c$

min. 3 ?
min. 5 ?
min. 8 ?
min. 11 ?
...

In Anlehnung an
Glenford J. Myers:
Methodisches Testen
von Programmen.
7. Auflage 2001

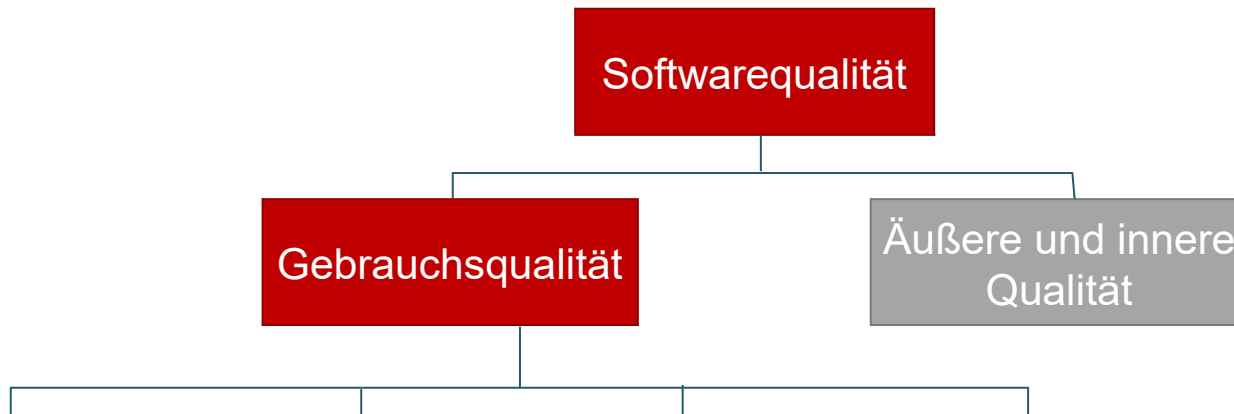
Wie viele / welche Fallunterscheidungen ergibt dies ?



Software-Qualität nach ISO/IEC 9126 (1)

Softwarekons
WS 2016/17

Wiederholung
SWT



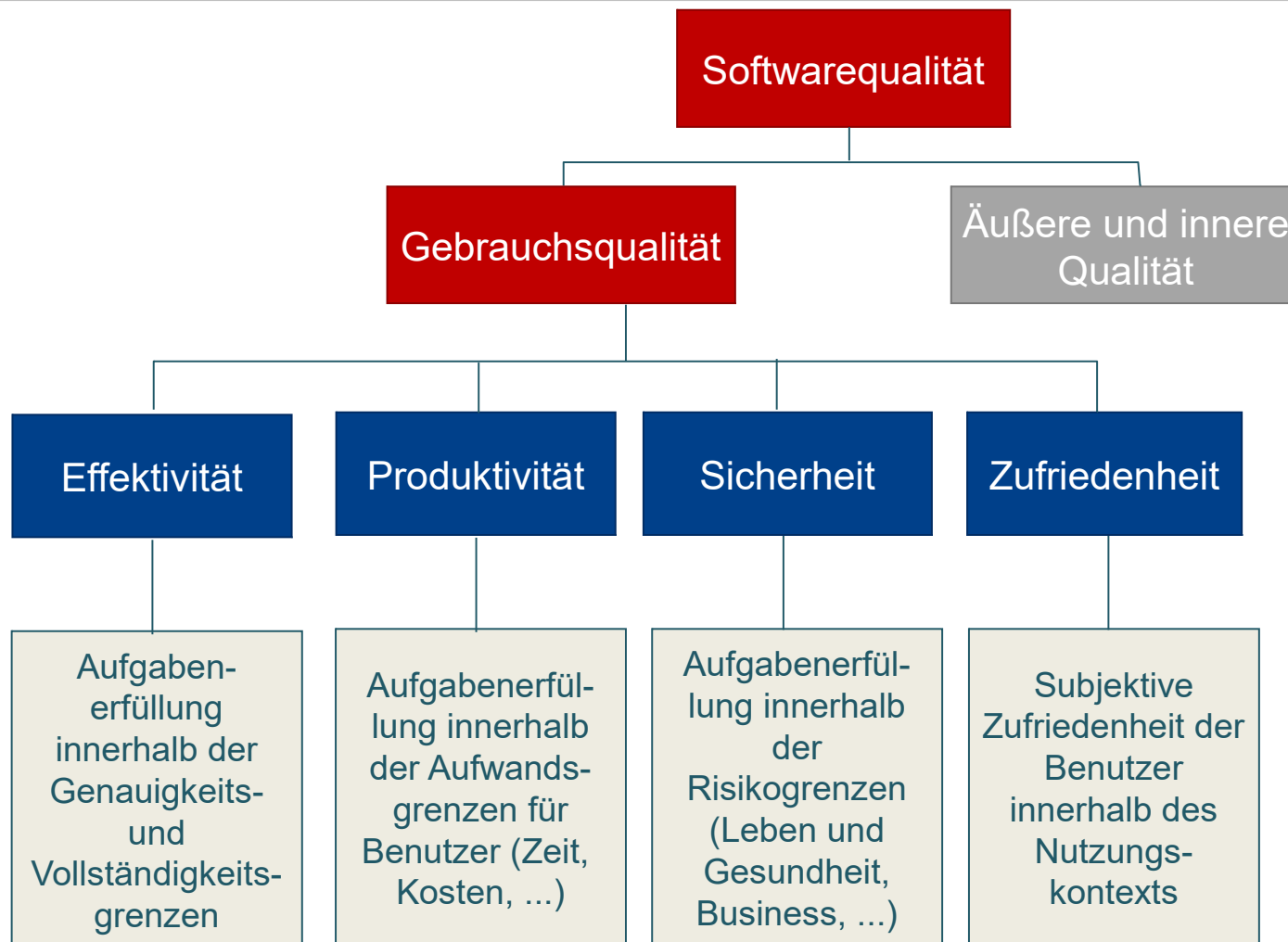
Welche Software-Qualitätseigenschaften kennen Sie ?
(Wiederholung von SWT)

ISO/IEC 9126: Bewerten von Softwareprodukten, Qualitätsmerkmale und Leitfaden zu ihrer Verwendung. (Seit 2005 ersetzt durch ISO/IEC 25000 Software engineering – Software product Quality Requirements and Evaluation (SQuaRE).)

Software-Qualität nach ISO/IEC 9126 (1)

Softwarekons
WS 2016/17

Wiederholung
SWT

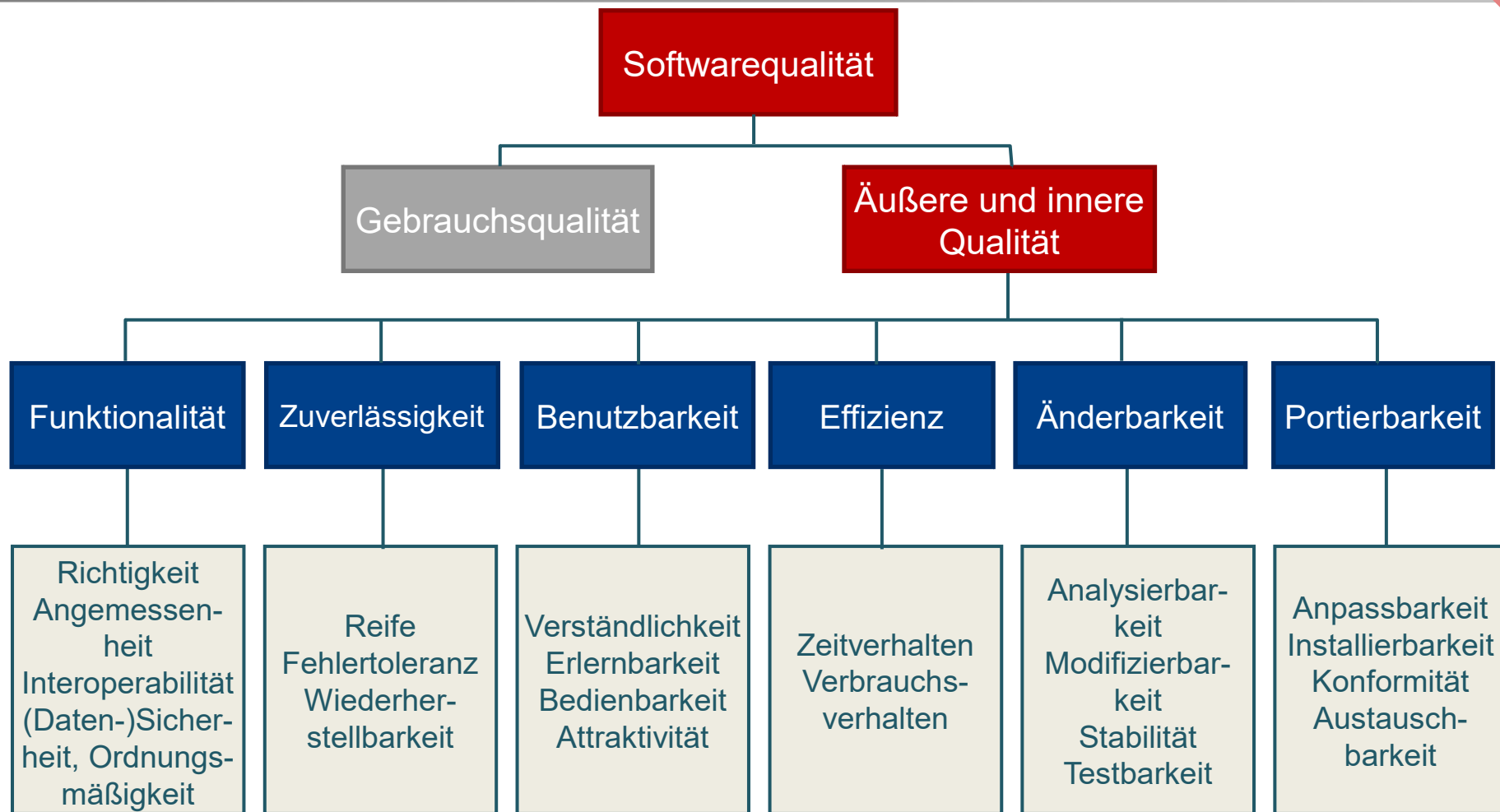


ISO/IEC 9126: Bewerten von Softwareprodukten, Qualitätsmerkmale und Leitfaden zu ihrer Verwendung. (Seit 2005 ersetzt durch ISO/IEC 25000 Software engineering – Software product Quality Requirements and Evaluation (SQuaRE).)

Software-Qualität nach ISO/IEC 9126 (2)

Softwarekons
WS 2016/17

Wiederholung
SWT



ISO/IEC 9126: Bewerten von Softwareprodukten, Qualitätsmerkmale und Leitfaden zu ihrer Verwendung. (Seit 2005 ersetzt durch ISO/IEC 25000 Software engineering – Software product Quality Requirements and Evaluation (SQuaRE).)

Welche Qualitätsmerkmale können Ihrer Ansicht nach im Konflikt zueinander stehen ?

Effektivität

Produktivität

Sicherheit

Zufriedenheit

Funktionalität

Zuverlässigkeit

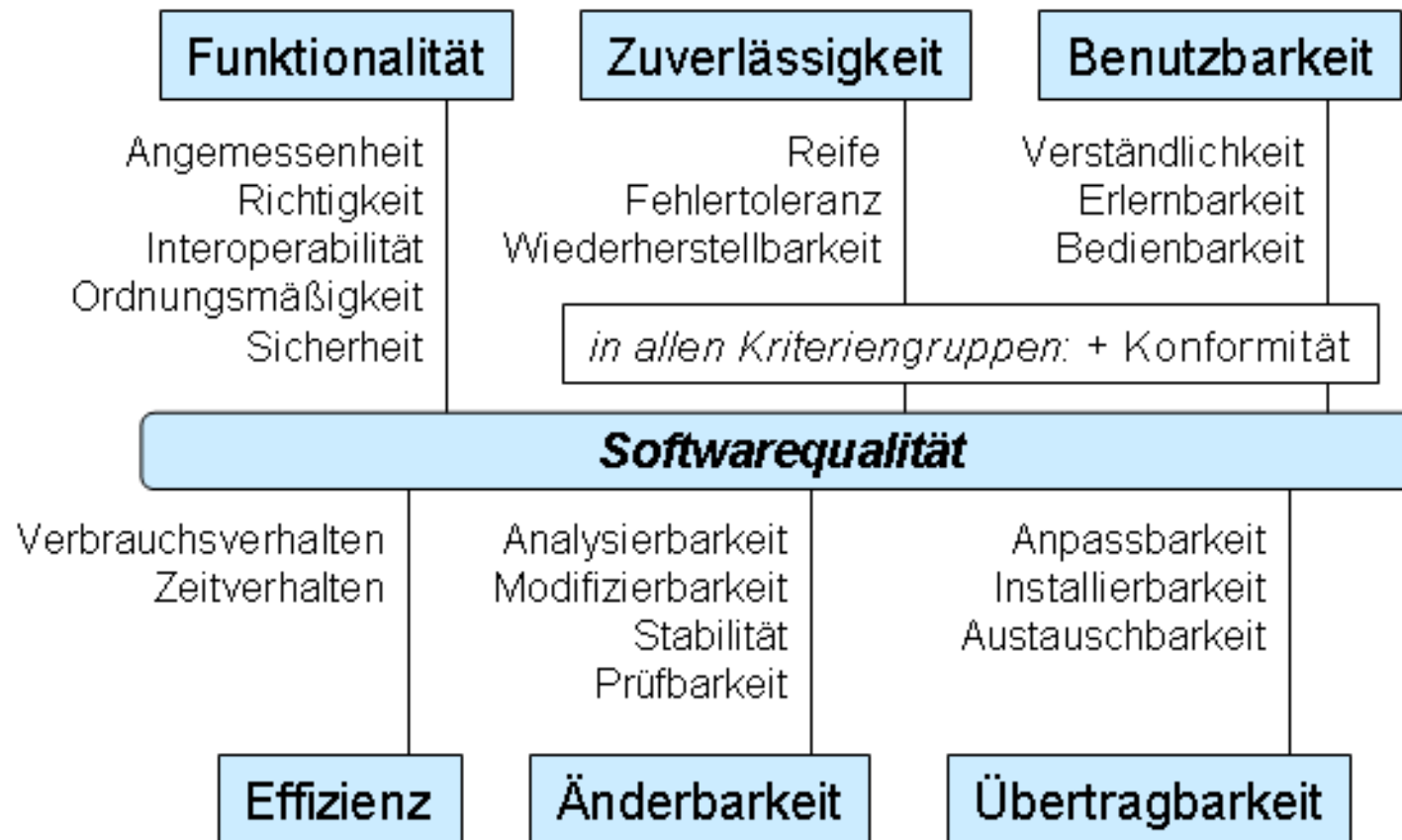
Benutzbarkeit

Effizienz

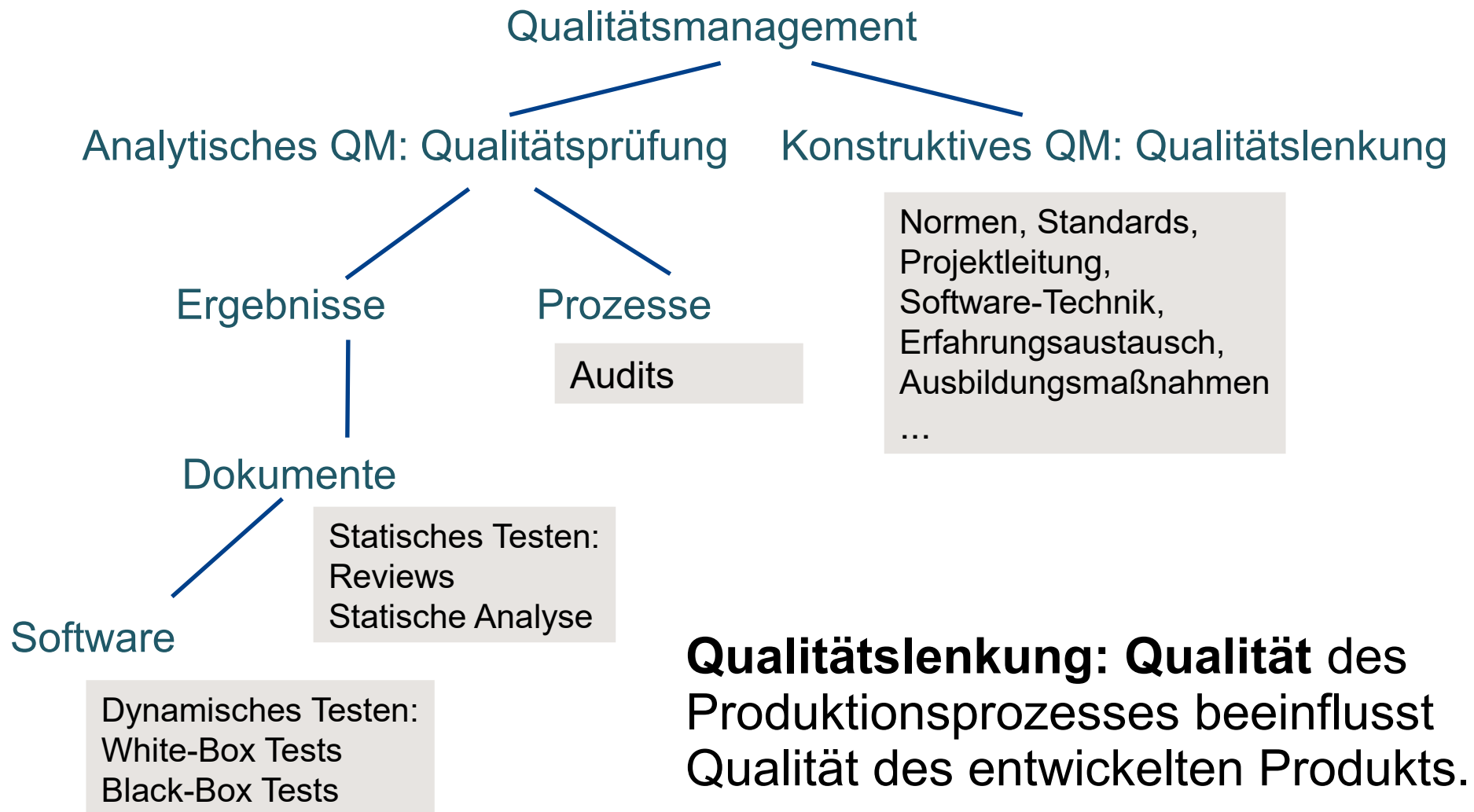
Änderbarkeit

Portierbarkeit

Qualitätsmerkmale von Softwaresystemen (ISO 9126)

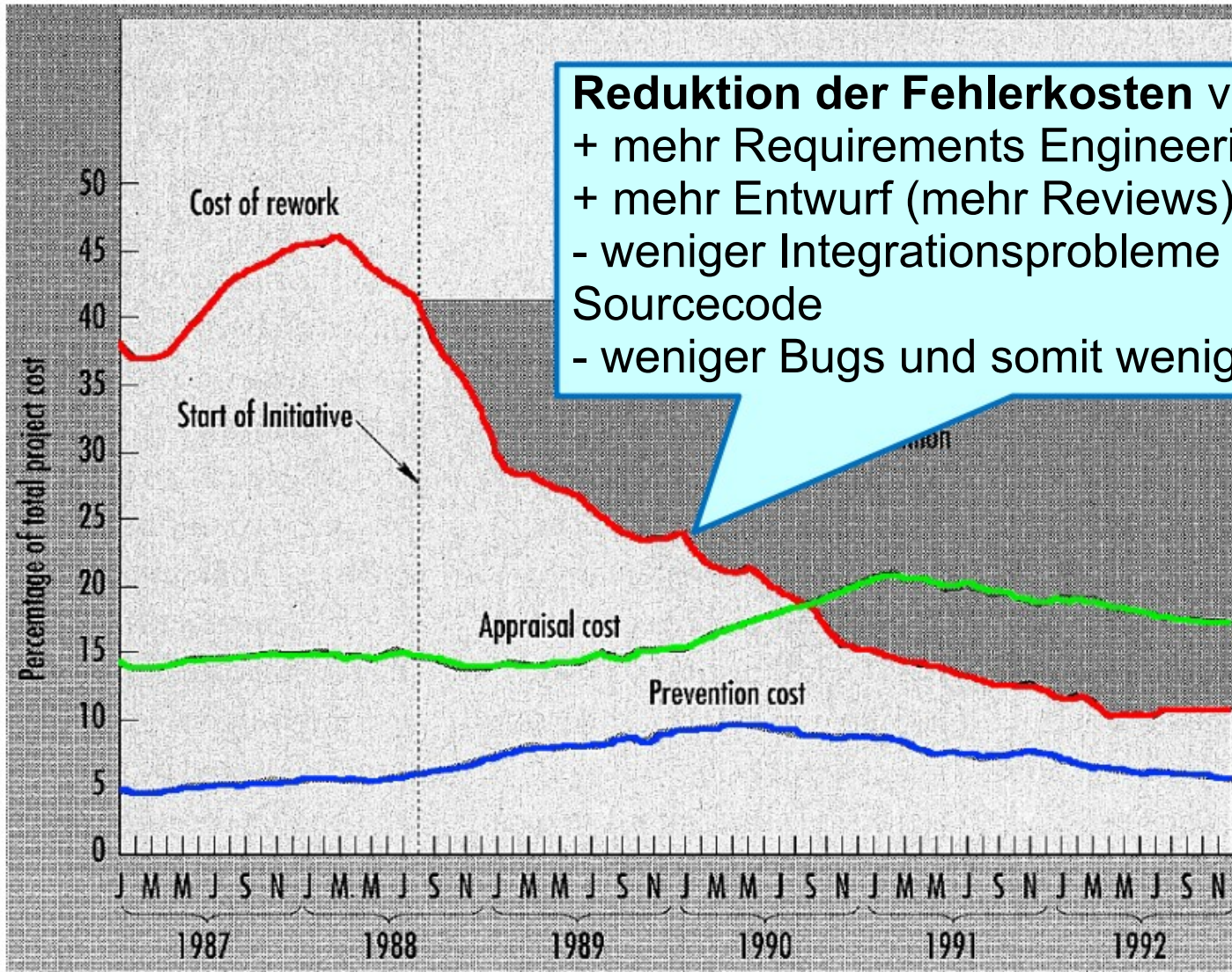


Quelle: Wikimedia



Was bringt Prozessverbesserung ?

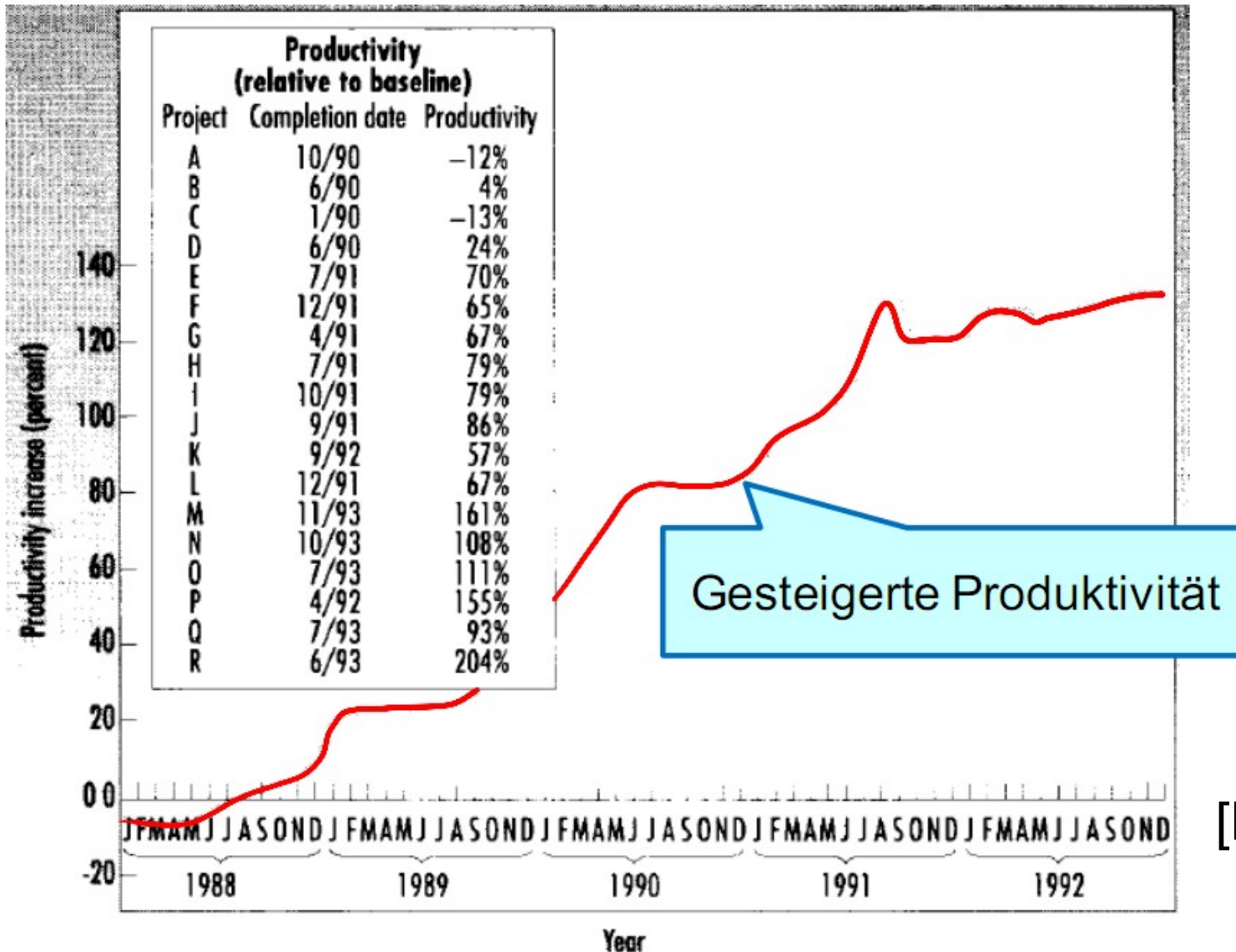
Reduktion der Fehlerkosten



[Dion 1993]

Was bringt Prozessverbesserung ?

Gesteigerte Produktivität



Zeitpunkt der Fehlerfindung

Zeitpunkt der Fehlerentstehung

	Anforderungen	Architektur	Konstruktion	System Test	Post-release
Anforderungen	1x	3x	5-10x	10x	10-100x
Architektur	-	1x	10x	15x	25-100x
Konstruktion	-	-	1x	10x	10-25x

Quelle: Diverse

Qualitätslenkung: Konstruktive Maßnahmen

- **Fehlerverhindernde** / fehlervermeidende Prozesse definieren.
- Prüf- und **Korrekturverfahren** in Prozesse integrieren.
- Prüfergebnisse zur Verbesserung des Prozesses verwenden.
- Keine systematische, ingenieurmäßige Vorgehensweise, welche Erreichung gegebener Qualitätsanforderungen garantiert.
- Konstruktive Maßnahmen, um generelles **Qualitätsniveau** zu heben.
- **Rigoreuse Qualitätsprüfung** (und Behebung der festgestellten Mängel):
 - Mittel zur Sicherstellung konkreter Qualitätsanforderungen an Software, während aller Phasen der Entwicklung.



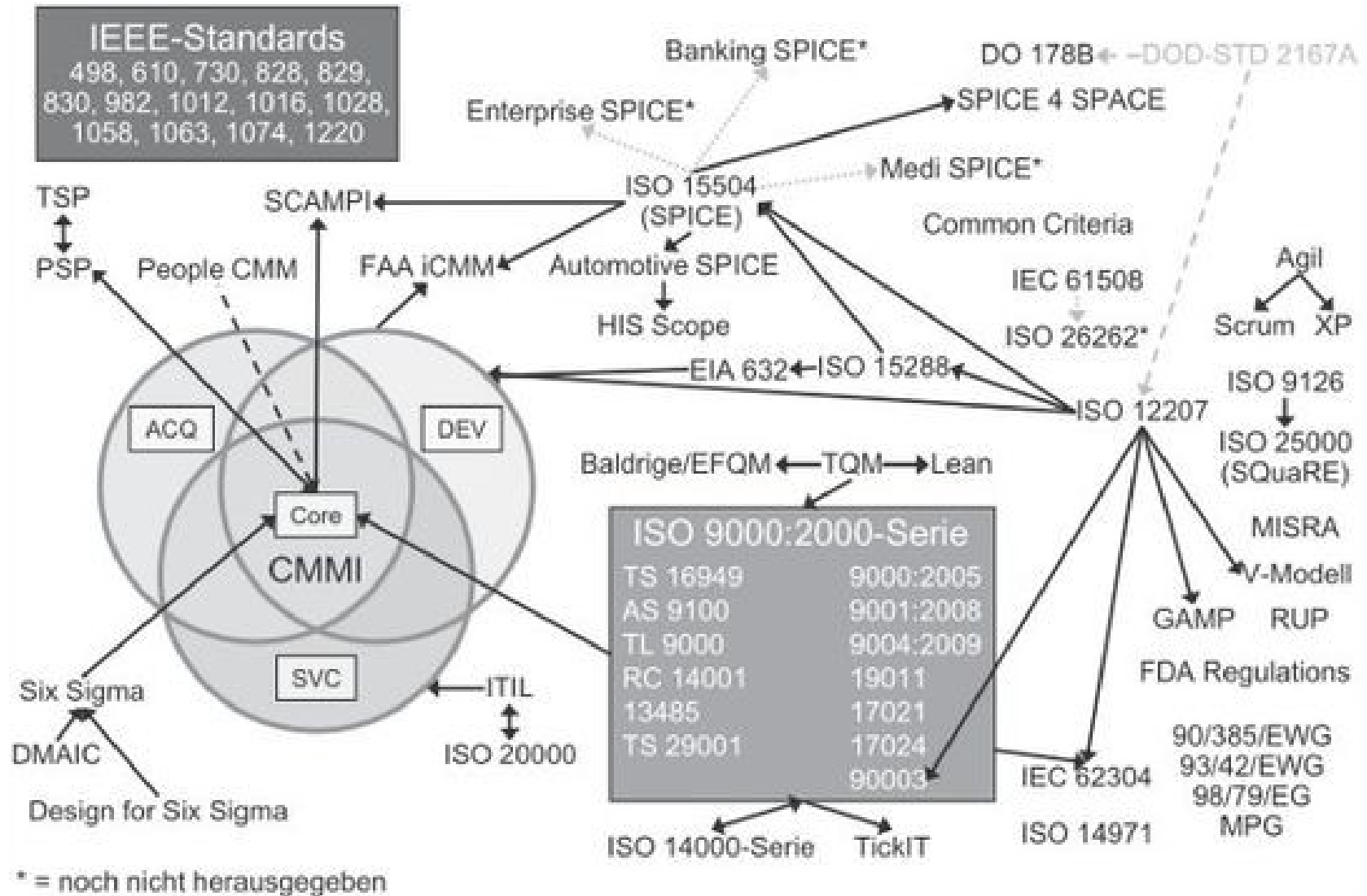
- Gruppe von **Qualitätsprüfern** prüft teilweise oder ganz Softwaresystem und dazugehörige Dokumentation.
- Code, Entwurf, Spezifikationen, Testentwürfe, Standards, etc. können überprüft werden.
- Um fortzufahren Software oder Dokumente als freigegeben markieren.
- Drei verschiedene Prüfungen mit verschiedenen Zielen:
 - Inspektion zur **Mängelbeseitigung** (Produkt).
 - Prüfung der **Fortschrittsbewertung** (Produkt / Abläufe).
 - Prüfung der **Qualität** (Produkt / Standards).



- **Qualität** des Produktionsprozesses beeinflusst Qualität des entwickelten Produkts.
- Wichtig in Softwareentwicklung, weil einige **Produktqualitätseigenschaften** schwer zu beurteilen sind.
- **Komplexe Beziehung** zwischen Softwareprozess und Produktqualität.
 - Anwendung von **individuellen Fähigkeiten** und **Erfahrungen**: Wichtig in Softwareentwicklung.
 - **Externe Faktoren** (*Neuartigkeit der Anwendung oder beschleunigte Entwicklungszeitpläne*) können Produktqualität beeinträchtigen.

- Prüfung der Produkte:
 - Statische Prüfung:
 - Review
 - Statische Analyse
 - Formale Programmverifikation
 - Model Checking
 - Dynamische Prüfung:
 - Test
 - Simulation
 - Prototypen
- Prüfung der Prozesse:
 - Audits
 - Prozessbeurteilung

Zoo der Qualitätsstandards

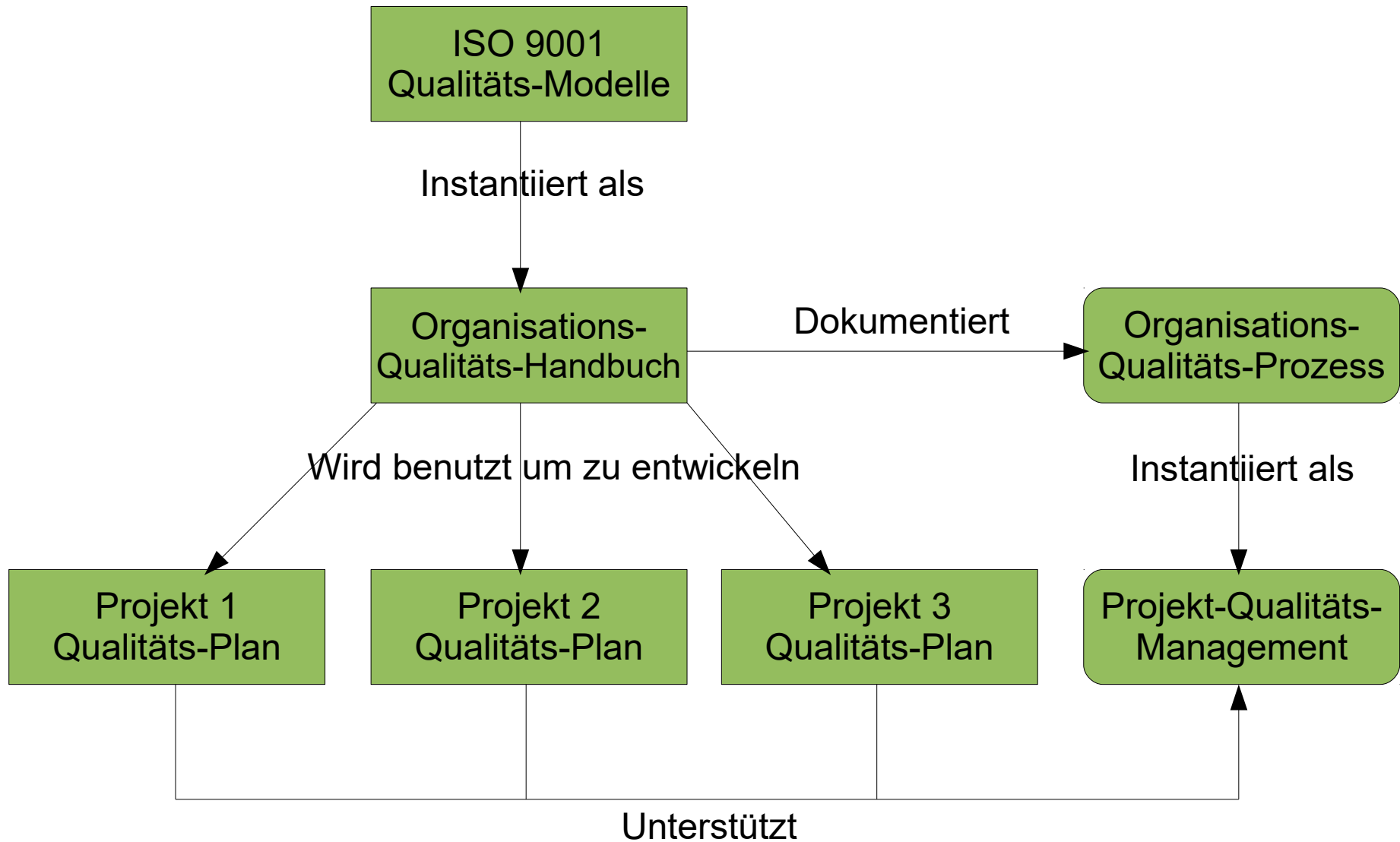


Quelle: E.Wallmüller, Software Quality Engineering, Hanser 2011, S. 30

- Internationale Sammlung von Standards.
→ Als Basis des Qualitätsmanagements verwendbar.
- 1987 eingeführt.

Grundlage ISO 9000:

- ISO 9000 „**Qualitätsmanagementsysteme - Grundlagen und Begriffe**“
- Erläutert Grundlagen für Qualitätsmanagementsysteme und in Normenreihe ISO 900x verwendete Begriffe.
- Erklärt **prozessorientierten Ansatz** des Qualitätsmanagements.
- Aktuelle Version von 2015 (ISO 9000:2015).



- **Unternehmens-Qualitätshandbuch** sollte Qualitätsstandards und Abläufe enthalten.
- Externe Stelle muss Konformität des Qualitätshandbuchs mit ISO 9001 bestätigen.
- Manche Kunden verlangen **ISO 9001-Konformität** von Anbietern.
- Weltweit mehr als 1 Millionen Organisationen ISO 9001-zertifiziert.
- Trotzdem als **aufwendig** und nicht für alle Unternehmen gleichermaßen geeignet kritisiert.

Diskussionsfrage: ISO 9001 vs. Produktqualität

Softwarekonstruktion
WS 2016/17



- Mit ISO 9001 Qualität eines Softwareprodukts zertifizierbar ?
- **Antwort:**

Wurzeln:

- Systematische **Prozessverbesserung** (Deming 1986).
- Prozessorientierte Software-Entwicklung (Humphrey 1989).
- Beurteilung des **Reifegrads** der Prozesse eines Software-Lieferanten.
- **Capability Maturity Model (CMM)** (Paulk et al. 1993).
- Modelle als Grundlage und Gerüst für Prozessverbesserung:
 - CMM.
 - SPICE – Software Process Improvement and Capability Determination (ISO/IEC 15504).
- CMM spezialisiert für Systeme, Leute, Beschaffung,...
- Entwicklung eines umfassenden, zuschneidbaren Rahmenmodells:
CMMI (Capability Maturity Model Integrated).

0 – Incomplete

Die Arbeit wird so durchgeführt, dass die fachlichen Ziele (in CMMI "Specific Goals" genannt, z. B. bei der Projektplanung ein Projektplan) nicht erreicht werden.

1 – Performed

Die Arbeit wird so durchgeführt, dass die fachlichen Ziele erreicht werden.

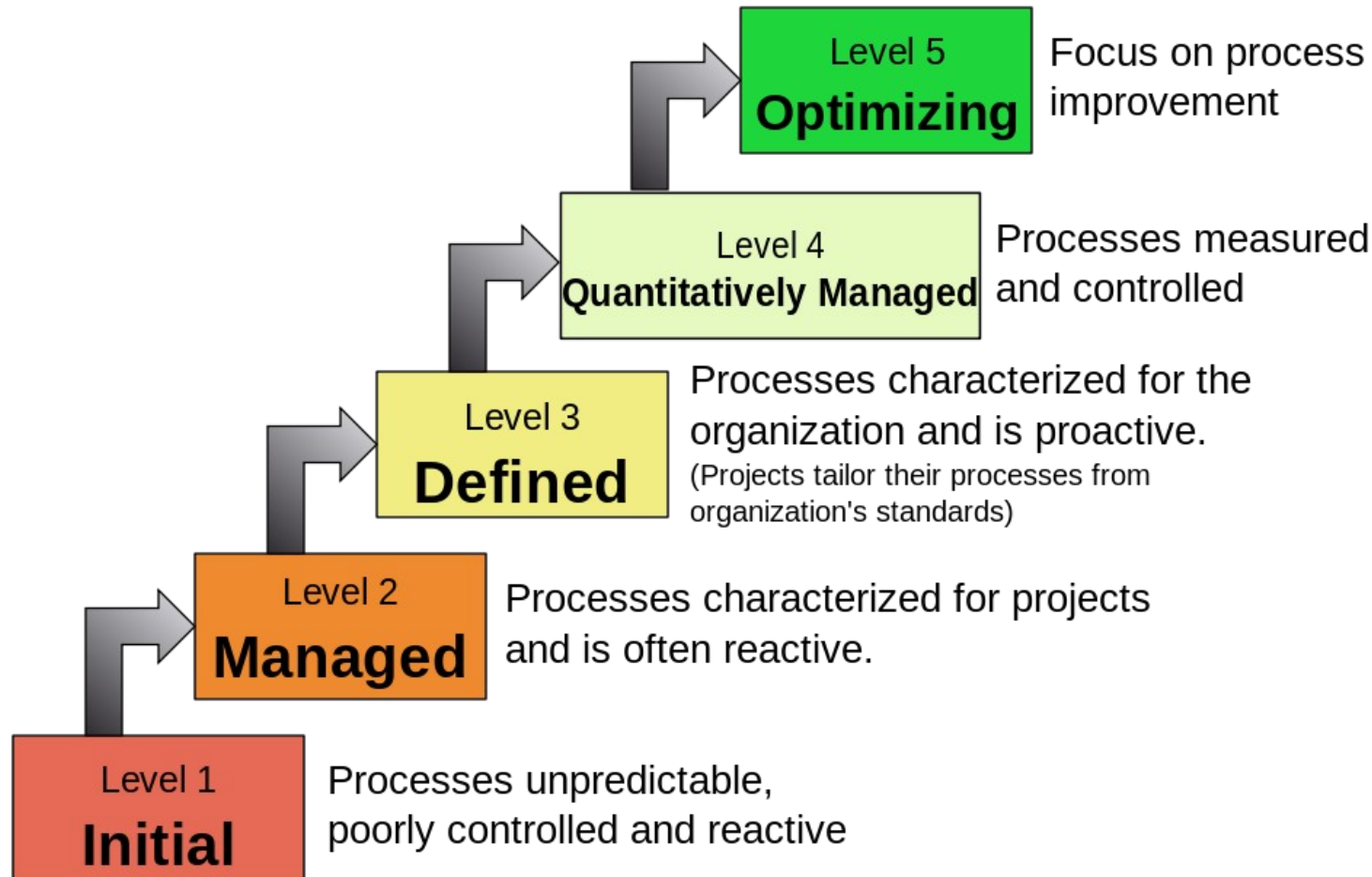
2 – Managed

Die Arbeit wird gelenkt.

3 – Defined

Die Arbeit wird mit Hilfe eines angepassten Standardprozesses durchgeführt und die Arbeitsweise verbessert.

Characteristics of the Maturity levels



Quelle: CMMI Standard

1 – Initial

Keine Anforderungen. Diesen Reifegrad hat jede Organisation automatisch.

2 – Managed

Die Projekte werden geführt. Ein ähnliches Projekt kann erfolgreich wiederholt werden.

3 – Defined

Die Projekte werden nach einem angepassten Standardprozess durchgeführt und es gibt eine organisationsweite kontinuierliche Prozessverbesserung.

4 – Quantitatively Managed

Es wird eine statistische Prozesskontrolle durchgeführt.

5 – Optimizing

Die Arbeit und Arbeitsweise werden mit Hilfe einer statistischen Prozesskontrolle verbessert.

CMMI 1.3 Prozessgebiete

Prozessgebiet (engl.)	Prozessgebiet (dt.)	Kategorie	Reifegrad
Organizational Process Focus (OPF)	Organisationsweiter Prozessfokus	Process Management	3
Organizational Process Performance (OPP)	Organisationsweite Prozessfähigkeit	Process Management	4
Organizational Training (OT)	Organisationsweites Training	Process Management	3
Product Integration (PI)	Produktintegration	Engineering	3
Project Monitoring and Control (PMC)	Projektverfolgung und -steuerung	Project Management	2
Project Planning (PP)	Projektplanung	Project Management	2
Process and Product Quality Assurance (PPQA)	Qualitätssicherung von Prozessen und Produkten	Support	2
Quantitative Project Management (QPM)	Quantitatives Projektmanagement	Project Management	4

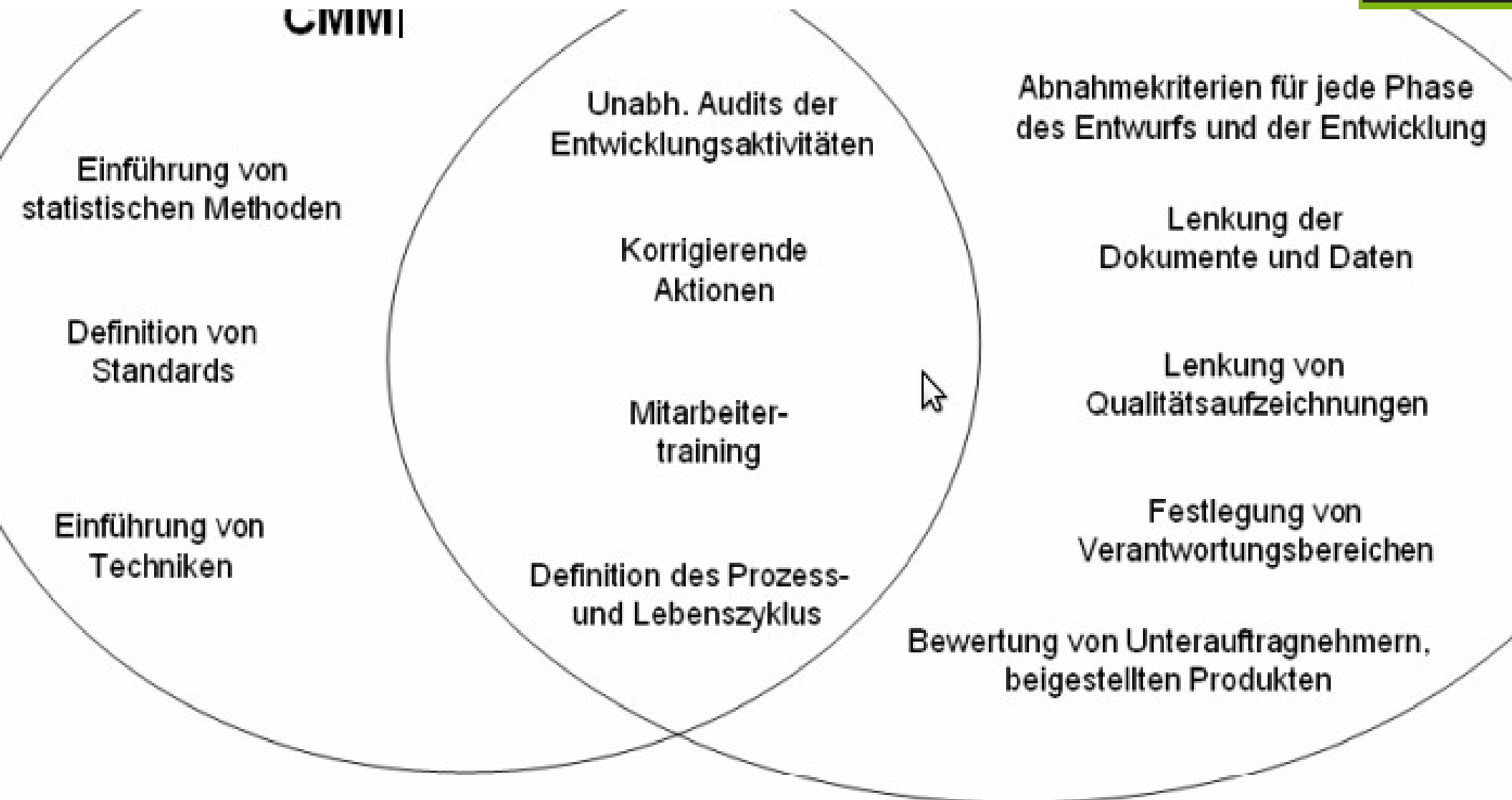
CMMI 1.3 Prozessgebiete

Prozessgebiet (engl.)	Prozessgebiet (dt.)	Kategorie	Reifegrad
Causal Analysis and Resolution (CAR)	Ursachenanalyse und Problemlösung	Support	5
Configuration Management (CM/SCM)	Konfigurationsmanagement	Support	2
Decision Analysis and Resolution (DAR)	Entscheidungsanalyse und -findung	Support	3
Integrated Project Management (IPM)	Integriertes Projektmanagement	Project Management	3
Measurement and Analysis (MA)	Messung und Analyse	Support	2
Organizational Performance Management (OPM)	Organisationsweites Prozessfähigkeitsmanagement	Process Management	5
Organizational Process Definition (OPD)	Organisationsweite Prozessdefinition	Process Management	3

Prozessgebiet (engl.)	Prozessgebiet (dt.)	Kategorie	Reifegrad
Requirements Development (RD)	Anforderungsentwicklung	Engineering	3
Requirements Management (REQM)	Anforderungsmanagement	Project Management	2
Risk Management (RSKM)	Risikomanagement	Project Management	3
Supplier Agreement Management (SAM)	Management von Lieferantenvereinbarungen	Project Management	2
Technical Solution (TS)	Technische Umsetzung	Engineering	3
Validation (VAL)	Validierung	Engineering	3
Verification (VER)	Verifizierung	Engineering	3

CMMI vs. ISO 9001

Softwarekonstruktion
WS 2016/17



Ist CMM Stufe 3 erreicht, ist wahrscheinlich noch einiges zu tun, um ISO 9001 zu erreichen, da einige Bereiche durch CMM nicht abgedeckt werden.

Ist ISO 9001 erfüllt, dann gibt CMM zusätzlich Hilfestellung insbesondere auf den Gebieten der Technik, Prozessdefinition und Metriken.

In diesem Abschnitt:

- **Software-Fehler** können Menschen, Maschinen, Unternehmen etc. Schaden zufügen.
- „**Austesten**“ eines Programms in der Regel **nicht möglich**.
- **Software-Qualitätsmanagement** soll sicherstellen, dass Produkt wenig Fehler hat.
- Beinhaltet, dass
 - für Abläufe und Produkte Standards erstellt werden
 - bei diesen Abläufen auch Standards eingehalten werden.

Im nächsten Abschnitt:

- Grundlagen der **Softwareverifikation**: Fehlerarten.

- Mehr Informationen für einige der behandelten Begriffe; zum Nachlesen zu Hause.

Motte im Rechner Mark II verursacht Fehler in Relay Nr. 70,
Panel F.

Mrs. Grace Murray Hopper beseitigt Fehler, dokumentiert im
Log-Buch:

»First actual case
of bug being found.«

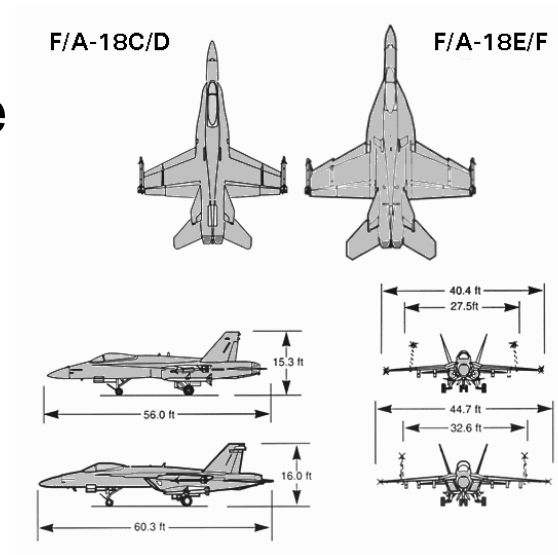
- »offen«-sichtlicher Fehler.
- **Beseitigung ist einfach.**



F-18 am Äquator: Dokumentationsfehler

US Air Force, Programm zur Raketensteuerung

- Aus Speicherplatzmangel **keine Neuberechnung** der Flugkoordinaten.
- Nur **Änderung** des Vorzeichens.
- **Programm** unverändert in Autopiloten des Jäger F-18 **übernommen**.
- Beim Überflug des Äquator drehte sich Maschine auf den Kopf wie zuvor!
- **Glück im Unglück:** Man bemerkte dies im Simulator ...



September 2004: Drei Wochen Verspätung

Durch anhaltende Probleme mit der Computersoftware können die Arbeitsagenturen erst drei Wochen später als geplant die Antragsdaten für das Arbeitslosengeld II flächendeckend erfassen. Ursprünglich sollte das Programm schon ab 4. Oktober 2004 zur Verfügung stehen.

Mittlerweile haben sich die Bundesagentur für Arbeit (BA) und die Software-Entwickler der Telekom-Tochter T-Systems geeinigt: Das Programm soll stufenweise eingeführt werden. "Wir sind zurzeit in der Testphase, um Fehler zu ermitteln und zu beheben. Am 4. Oktober werden wir planmäßig starten und das System stufenweise hochfahren", sagte ein Sprecher von T-Systems im Gespräch mit **wdr.de** am Freitag (10.09.04). Das Computerprogramm werde Anfang Oktober in einigen Agenturen anlaufen, damit die BA Erfahrungen sammeln kann.



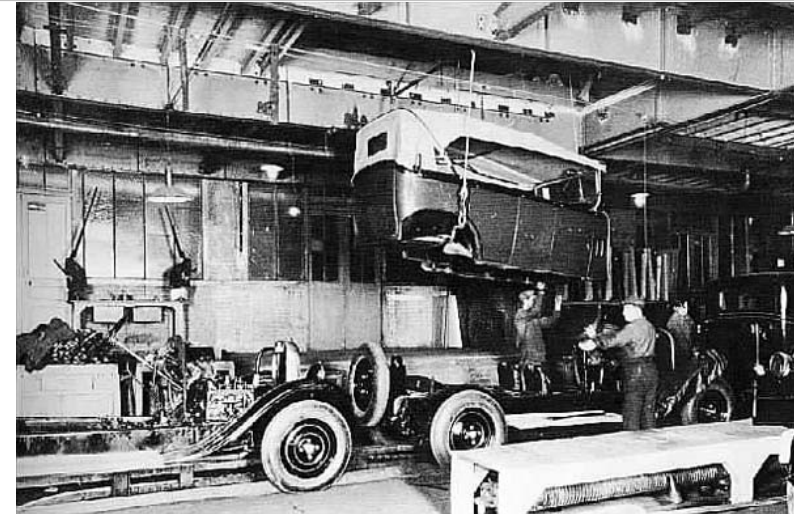
In einigen Agenturen wird das Programm getestet.

September 2005: Durch einen Fehler in der Software A2LL kommt es derzeit zu falschen Krankenkassen-Meldungen, teilt die Bundes-agentur für Arbeit (BA) mit. In mehreren hunderttausend Fällen seien Meldungen zur Krankenversicherung, also Anmeldungen, Abmeldungen, Veränderungsmitteilungen, von Arbeitslosengeld-II-Empfängern ohne Grund automatisch storniert worden.

<http://www.heise.de/newsticker/meldung/62595>

Automobil-Industrie vs. Softwaretest

- Eingangstest der Komponenten
(= **Komponententest**)
- Laufende Zwischenkontrollen
am Fließband (= **Integrationstest**)
- Realitätsnaher Einsatztest
(= **Systemtest**)
- Probefahrt des Kunden
(= **Abnahmetest**)
- Renneinsatz
(= **Performanztest**, Lasttest)
(Stabilität, Zuverlässigkeit, Robustheit)
- Crashtest (= **Stresstest**)



- Grad, in dem ein System, eine Komponente oder ein Prozess die Kundenerwartungen und -bedürfnisse erfüllt.
[nach IEEE 610: „Standard Glossary of Software Engineering Terminology“]
- Qualitätsanforderungen geben vor, welche **Qualitätsmerkmale** das Produkt in welcher **Güte** aufweisen soll (Qualitätsniveau).
- Gesamtheit aller Qualitätsmerkmale und deren geforderte Ausprägung.
- Qualitätsmerkmale beziehen sich auf Anforderungen:
 - **Funktionale Anforderungen**
(Fachlichkeit, Funktionen, Schnittstellen, ...).
 - **Nicht-funktionale Anforderungen**
(Qualitäts- und Realisierungsanforderungen, Projektspezifische Anforderungen, ...).

Vorhandensein von Funktionen mit festgelegten Eigenschaften. Diese Funktionen erfüllen die festgelegten oder vorausgesetzten Anforderungen.

Angemessenheit

- Merkmale von Software, die sich auf das Vorhandensein und die Eignung einer Menge von Funktionen für spezifizierte Aufgaben beziehen.
- Die Fähigkeit eines Softwareprodukts für spezifizierte Aufgaben und Zielsetzungen der Benutzer einen geeigneten Satz Funktionen zu liefern.

Richtigkeit

- Merkmale von Software, die sich auf das Liefern der richtigen oder vereinbarten Ergebnisse oder Wirkungen beziehen.
- Die Fähigkeit eines Softwareprodukts, die richtigen oder vereinbarten Ergebnisse oder Wirkungen mit dem benötigten Grad an Genauigkeit zu liefern.

Interoperabilität

- Merkmale von Software, die sich auf ihre Eignung beziehen, mit vorgegebenen Systemen zusammenzuwirken.
- Die Fähigkeit eines Softwareprodukts, mit einer oder mehreren spezifizierten Komponenten zusammenzuwirken.

Ordnungsmäßigkeit

- Merkmale von Software, die bewirken, dass die Software anwendungsspezifische Normen oder Vereinbarungen oder gesetzliche Bestimmungen und ähnliche Vorschriften erfüllt.

Sicherheit

- Merkmale von Software, die sich auf ihre Eignung beziehen, unberechtigten Zugriff, sowohl versehentlich als auch vorsätzlich, auf Programme und Daten zu verhindern.
- Eigenschaften der Software, die sich auf die Fähigkeit beziehen, nicht autorisierte Zugriffe auf Programme oder Daten zu verhindern, unabhängig davon, ob diese versehentlich oder vorsätzlich erfolgen.

Merkmale, die sich auf die Fähigkeit der Software beziehen, ihr Leistungsniveau unter festgelegten Bedingungen über einen festgelegten Zeitraum zu bewahren.

Reife

- Merkmale von Software, die sich auf die Häufigkeit von Versagen durch Fehlzustände in der Software beziehen.
- Die Fähigkeit eines Softwareprodukts, Fehlerwirkungen aufgrund von Fehlerzuständen in der Software zu vermeiden.

Fehlertoleranz

- Merkmale von Software, die sich auf ihre Eignung beziehen, ein spezifiziertes Leistungsniveau bei Software-Fehlern oder Nicht-Einhaltung ihrer spezifizierten Schnittstelle zu bewahren.
- Die Fähigkeit eines Softwareprodukts, ein spezifiziertes Leistungsniveau auch bei Fehlfunktionen oder trotz Fehleingaben (z.B. falsche Bedienung) aufrecht zu erhalten.

Wiederherstellbarkeit

- Merkmale von Software, die sich beziehen auf die Möglichkeit, bei einem Versagen ihr Leistungsniveau wiederherzustellen und die direkt betroffenen Daten wiederzugewinnen und auf die dafür benötigte Zeit und den benötigten Aufwand.
- Die Fähigkeit eines Softwareprodukts, bei einer Fehlerwirkung das spezifizierte Leistungsniveau des Systems wiederherzustellen und die direkt betroffenen Daten wiederzugewinnen.

Merkmale, die sich auf den zur Benutzung erforderlichen Aufwand beziehen, und auf die individuelle Bewertung einer solchen Benutzung durch eine festgelegte oder vorausgesetzte Gruppe von Benutzern.

Verständlichkeit

- Merkmale von Software, die sich auf den Aufwand für den Benutzer beziehen, das Konzept und die Anwendung zu verstehen.
- Die Fähigkeit eines Softwareprodukts, den Benutzer in die Lage zu versetzen zu verstehen, ob die Software geeignet ist, und wie sie für eine bestimmte Aufgabe und Benutzungsbedingungen brauchbar ist.

Erlernbarkeit

- Merkmale von Software, die sich auf den Aufwand für den Benutzer beziehen, die Anwendung zu erlernen.
- Die Fähigkeit eines Softwareprodukts, einem Benutzer das Erlernen der Anwendung leicht zu machen.

Bedienbarkeit

- Merkmale von Software, die sich auf den Aufwand für den Benutzer bei der Bedienung und Ablaufsteuerung beziehen.

Merkmale, die sich auf das Verhältnis zwischen dem Leistungsniveau der Software und dem Umfang der eingesetzten Betriebsmittel unter festgelegten Bedingungen beziehen.

- **Zeitverhalten**

Merkmale von Software, die sich beziehen auf die Antwort- und Verarbeitungszeiten und auf den Durchsatz bei der Ausführung ihrer Funktionen.

- **Verbrauchsverhalten**

Merkmale von Software, die sich darauf beziehen, wie viele Betriebsmittel bei der Erfüllung ihrer Funktionen benötigt werden und wie lange.

Merkmale, die sich auf den Aufwand für Durchführung vorgegebener Änderungen beziehen.

Analysierbarkeit

- Merkmale von Software, die sich auf den Aufwand beziehen, der notwendig ist, um Mängel oder Ursachen von Versagen zu diagnostizieren oder um änderungsbedürftige Teile zu bestimmen.
- Die Fähigkeit eines Softwareprodukts, die Diagnose von Mängeln oder Ursachen von Fehlerwirkungen zu ermöglichen oder änderungsbedürftige Teile zu bestimmen.

Modifizierbarkeit

- Merkmale von Software, die sich auf den Aufwand beziehen, der zur Ausführung von Verbesserungen, zur Fehlerbeseitigung oder zur Anpassung an Umgebungsänderungen notwendig ist.
- Die Fähigkeit eines Softwareprodukts, die Durchführung spezifizierter Änderungen zu ermöglichen.

Stabilität

- Merkmale von Software bezüglich des Risikos unerwarteter Wirkungen von Änderungen.
- Fähigkeit eines Softwareprodukts, unerwartete Auswirkungen von Änderungen zu vermeiden.

Testbarkeit

- Merkmale von Software, die sich auf den Aufwand beziehen, der zum Testen der geänderten Software notwendig ist.
- Die Fähigkeit eines Softwareprodukts für einen Test nach einer Änderung.

Merkmale, die sich auf die Eignung der Software beziehen, von einer Umgebung in eine andere übertragen zu werden.

Anpassbarkeit

- Merkmale von Software, die sich auf die Möglichkeit beziehen, sie an verschiedene festgelegte Umgebungen anzupassen, wenn nur Schritte unternommen oder Mittel eingesetzt werden, die für diesen Zweck für die betrachtete Software vorgesehen sind.
- Die Fähigkeit eines Softwareprodukts, dass sie auf verschiedene Laufzeitumgebungen angepasst werden kann und dabei nur die Anpassungen vorzunehmen sind, die genau diesem Zweck dienen.

Installierbarkeit

- Merkmale von Software, die sich auf den Aufwand beziehen, der zur Installation der Software in einer festgelegten Umgebung notwendig ist.
- Die Fähigkeit eines Softwareprodukts, in einer spezifizierten Umgebung installierbar zu sein.

Konformität

- Merkmale von Software, die bewirken, dass die Software Normen oder Vereinbarungen zur Übertragbarkeit erfüllt.
- Die Fähigkeit eines Softwareprodukts, anwendungsspezifische Normen oder Vereinbarungen oder gesetzliche Bestimmungen und ähnliche Vorschriften zu erfüllen.

Austauschbarkeit

- Merkmale von Software, die sich beziehen auf die Möglichkeit, diese anstelle einer spezifizierten anderen Software in der Umgebung jener Software zu verwenden, und auf den dafür notwendigen Aufwand.
- Die Fähigkeit eines Softwareprodukts an Stelle einer anderen spezifizierten Software zum selben Zweck in der gleichen Umgebung genutzt zu werden.

- Nicht alle **Qualitätsmerkmale** lassen sich gleichzeitig gleich gut erfüllen.
- **Qualitätsplan** sollte für zu entwickelnde Software wichtigste Qualitätsmerkmale herausstellen.
- Prioritäten festlegen:
 - In engster Absprache mit Auftraggebern und Anwendern.
 - **Qualitätsanforderungen:** Bestandteil der **nicht-funktionalen Anforderungen** im Pflichtenheft.
- Qualitätsplan sollte vorgeben, wie **Qualitätsbewertung** der Software ablaufen soll.

- Legt **Mindestanforderungen** an Qualitätsmanagementsystem fest, die Organisation erfüllen muss.
- Um Produkte und Dienstleistungen bereitstellen zu können, die **Kundenerwartungen** und behördliche Anforderungen erfüllen.
- ISO 9001 wird insbesondere herangezogen, um Softwareprodukte zu entwerfen, zu entwickeln und zu pflegen.
 - Beschreibung von allgemeinen **Qualitätsmerkmalen** und Qualität von Abläufen.
 - Darlegung von organisatorischen und prozeduralen Normen, die definiert und in einem **Qualitätshandbuch** niedergeschrieben werden sollten.

- **Beschreibt Modelle** zur Darlegung der Qualitätssicherung in Entwicklung, Produktion, Montage und Kundendienst.
- Legt
 - kein **Vorgehensmodell** (Phasenmodell) fest.
 - einmalig oder periodisch im Unternehmen durchzuführende und pro Projekt durchzuführende **Maßnahmen** fest.
- Verlangt
 - **Darlegung** der Phasen, Ergebnisse und der jeweiligen Qualitätssicherungsmaßnahmen (Verifizierung).
 - Maßnahmen zur Dokumentation der Projektabläufe.
- **Erfüllung der Vorgaben** durch Audits unabhängiger Zertifizierungsstelle.
- Regelmäßige **Überwachungs- bzw. Wiederholungsaudits**.
- ISO IEC 90003 (früher ISO 9000-3):
 - Richtlinie für Anwendung von ISO 9001 auf Entwicklung und Wartung von Software.

Produktentwicklungs- prozess

Geschäfts-
Akquisition

Design und
Entwicklung

Test

Produktion
und Auslieferung

Service und
Support

Unterstützende Prozesse

Unternehmens-
Management

Einkaufs-
Management

Inventur-
Management

Konfigurations-
Management